Tema 2:
Programació multifil

Setmana 4. Mètodes sincronitzats

Forma comunicació fils

Necessitat → Comunicació entre fils.

Solució → Compartir un objecte.

Plantegem un exemple en el qual dos fils compartiran un objecte de la classe Comptador.

Classe Comptador

Comptador.java

```
1  class Comptador {
2
3      private int c=0; //atribut comptador
4
5      //constructor
6      Comptador (int c) {
7          this.c=c;
8      }
9
10     public void incrementa(){
11         c=c+1;
12         }
13
14     public void decrementa(){
15         c=c-1;
16         }
17     public int getValor(){
18         return c;
19         }
20
21 }//COMPTADOR
```

La classe defineix un atribut comptador.

Tenim tres mètodes:

- **incrementa** (Augmenta valor en 1)
- **decrementa** (Decrementa valor en 1)
- **getValor** (Torna el valor).

El constructor assigna valor inicial al comptador.

FilA

FilA.java

```
1   class FilA extends Thread{
2       private Comptador comptador;
3
4       public FilA(String n,Comptador c){
5           setName(n);
6           comptador=c;
7       }
8
9       public void run(){
10          for (int j=0;j<300;j++) {
11              comptador.incrementa();//incrementa el comptador
12              try {
13                  sleep(100);
14              }catch (InterruptedException e) {}
15          }
16          System.out.println( getName() + " comptador val " + comptador.getValor());
17      }
18  }//FI FilA
```

FilB

FilB.java

```
1   class FilB extends Thread{
2       private Comptador comptador;
3
4       public FilB(String n,Comptador c){
5           setName(n);
6           comptador=c;
7       }
8
9       public void run(){
10          for (int j=0;j<300;j++) {
11              comptador.decrementa();//decrementa el comptador
12              try {
13                  sleep(100);
14              }catch (InterruptedException e) {}
15          }
16          System.out.println(getName() + " comptador val " + comptador.getValor());
17      }
18  }//FI FilB
```

Mètode principal (main)

CompartirInf1.java

```
1  public class CompartirInf1{
2      public static void main (String[] args){
3          Comptador compt=new Comptador(100); //cree comptador valor inicial 100
4          FilA a=new FilA("FilA",compt); //cree filA (incrementa) i li passe comptador
5          FilB b=new FilB("FilB",compt); //cree filB (decrementa) i li passe comptador
6          a.start(); //inicie filA
7          b.start(); //inicie filB
8      }
9  }
```

Executem I

```
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ javac contador.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ javac HiloA
HiloA2.java  HiloA.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ javac HiloA.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ javac HiloB.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ javac CompartirInf1.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos $ java CompartirInf1
```

```
ramon@nowhere:PSP_Setmana 4$ javac Comptador.java
ramon@nowhere:PSP_Setmana 4$ javac FilA.java
ramon@nowhere:PSP_Setmana 4$ javac FilB.java
ramon@nowhere:PSP_Setmana 4$ javac CompartirInf1.java
ramon@nowhere:PSP_Setmana 4$ java CompartirInf1
FilA comptador val 100
FilB comptador val 100

ramon@nowhere:PSP_Setmana 4$ time java CompartirInf1
TECLAT: NO FA RES
FilB comptador val 100
FilA comptador val 100

real    0m30,188s
user    0m0,115s
sys     0m0,046s
```

Al sistema li ha costat 30 segons executar el programa. A més a més, el valor del comptador no és el mateix sempre.

QUÈ HA PASSAT?

Executem II

Els dos fils accedixen a l'atribut "c" de Comptador i un va incrementant i l'altre decrementant la variable.

SOLUCIÓ:
Hem de sincronitzar l'accés als objectes comuns.

Accés objectes comuns

Operacions sobre objectes comuns (increment i decrement) s'han de fer de forma atòmica (fins que no acabe la suma no es realitza la resta).

Com ho aconseguim?

→ Afegim **synchronized** a la part de codi que volem que s'execute de forma atòmica.

synchronized

```
synchronized (object) {
// sentències crítiques
}
```

Cada vegada que un fil intenta accedir a un bloc sincronitzat li pregunta a aquest objecte si no hi ha cap altre fil que ja el tinga bloquejat. Si està ocupat, el fil es suspèn, si està lliure, l'agarra i el bloqueja fins que acaba la execució.

FilA synchronized

FilA.java

```
1  class FilA extends Thread{
2      private Comptador comptador;
3
4      public FilA(String n,Comptador c){
5          setName(n);
6          comptador=c;
7      }
8
9      public void run(){
10         synchronized(comptador){
11             for (int j=0;j<300;j++)
12                 comptador.incrementa();//incrementa el comptador
13
14             System.out.println( getName() + " comptador val " + comptador.getValor());
15         }
16     }
17 }//FI FilA
```

Modifiquem FilA. Afegim synchronized i eliminem captura d'interrupció.

FilB synchronized

FilB.java

```
1  class FilB extends Thread{
2      private Comptador comptador;
3
4      public FilB(String n,Comptador c){
5          setName(n);
6          comptador=c;
7      }
8
9      public void run(){
10         synchronized(comptador){
11             for (int j=0;j<300;j++)
12                 comptador.decrementa();//decrementa el comptador
13
14             System.out.println(getName() + " comptador val " + comptador.getValor());
15         }
16     }
17 }//FI FilB
```

Modifiquem run → Afegim synchronized i eliminem captura d'interrupció

Executem després de les modificacions

```
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos/sinchronized $ javac HiloA.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos/sinchronized $ javac HiloB.java
david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos/sinchronized $ javac CompartirInf1.java

david@david-OEM ~/pss/ut2/3 Comunicación y sincronización de hilos/sinchronized $ java CompartirInf1
HiloA contador vale 400
HiloB contador vale 100
```

```
ramon@nowhere:PSP_Setmana 4$ javac Comptador.java
ramon@nowhere:PSP_Setmana 4$ javac FilA.java
ramon@nowhere:PSP_Setmana 4$ javac FilB.java
ramon@nowhere:PSP_Setmana 4$ javac CompartirInf1.java
ramon@nowhere:PSP_Setmana 4$ java CompartirInf1
FilA comptador val 400
FilB comptador val 100
```

Atenció! Si accedeix primer B a la zona crítica el resultat serà:

```
ramon@nowhere:PSP_Setmana 4$ java CompartirInf1
FilB comptador val -200
FilA comptador val 100
```

Pràctica 8: Compte

Es facilita la plantilla de la classe Compte. Inclou un atribut saldo, un constructor i tres mètodes:

- un retornarà el valor del saldo.
- un altre restarà del saldo una quantitat.
- el tercer realitzarà les comprovacions per fer la retirada de diners.

Hi ha que completar els mètodes getSaldo i restar.

Classe Compte

Declaració de classe i constructor

Compte.java

```
1   class Compte{
2       private int saldo;
3
4       //constructor
5       Compte (int s)
6       {
7           saldo=s; //inicialitza el saldo actual
8       }//-------------------------------
```

Classe Compte

mètode RetirarDiners

```
21      // comprova que es puga retirar diners i els retira
22      void RetirarDiners(int cant, String nom) {
23          if (getSaldo() >= cant) {
24              System.out.println(nom + " va a retirar saldo (actual és: " + getSaldo() + ")");
25              try {
26                  Thread.sleep(1000);
27              } catch (InterruptedException ex) {
28              }
29              restar(cant);
30              System.out.println(nom + " retira => : " + cant + ". Saldo Actual: " + getSaldo() + "");
31          } else {
32              System.out.println(nom + " No pot retirar diners no hi ha saldo. Saldo Actual: " + getSaldo() + "");
33          }
34
35      }// fi retirars diners------------------------------
```

Classe Compte. Mètodes a desenvolupar

```
//torna el saldo
int getSaldo()
{
    //torna el saldo
}//-----------------------------

// resta la quantitat al saldo
void restar(int quantitat)
{

}//-----------------------------------
```

Mètodes a desenvolupar

Classe Persona

Creem una classe Persona que utilitzarà l'objecte Compte. Simulem accés concurrent a Compte mitjançant una classe que crearà fils que accediran a l'objecte Compte i compartiran informació.

Persona

```
3  class Persona extends Thread {
4      private Compte c; // declare objecte compte c
5      String nom;
6      // constructor
7      public Persona(String n, Compte c) {
8          super(n);
9          this.c = c;
10     }
11     // run
12     public void run() {
13         for (int x = 1; x <= 4; x++) {
14             c.RetirarDiners(10, getName());
15         }
16     }
17 }
```

Usa classe Compte per retirar els diners.

run realitza un bucle que retira diners (10€), i obté el nom del fil.

Classe UsPersonaCompte

```
3  public class UsPersonaCompte {
4      public static void main(String[] args) {
5          Compte c = new Compte(40);
6          Persona h1 = new Persona("Anna", c);
7          Persona h2 = new Persona("Joan", c);
8
9          h1.start();
10         h2.start();
11     }
12
13 }
```

Execució

```
[ramon@nowhere PSP_Setmana 4]$ java CompartirInf3
Es va a treure diners (saldo actual es: 40)
Es va a treure diners (saldo actual es: 40)
Anna treu => : 10. Saldo Actual: 30
Joan treu => : 10. Saldo Actual: 20
Es va a treure diners (saldo actual es: 20)
Es va a treure diners (saldo actual es: 20)
Anna treu => : 10. Saldo Actual: 10
Joan treu => : 10. Saldo Actual: 10
Es va a treure diners (saldo actual es: 10)
Es va a treure diners (saldo actual es: 10)
Anna treu => : 10. Saldo Actual: -10
Joan treu => : 10. Saldo Actual: -10
Anna No pot treure diners. No hi ha saldo. Saldo Actual: -10
Joan No pot treure diners. No hi ha saldo. Saldo Actual: -10
```

Observa que permet retirar saldo quan aquest era 0.

Mètodes sincronitzats I

- Cada **objecte Java** té associat un **lock** (bloqueig)
- La paraula "synchronized" pot utilitzar-se per a assenyalar aquells mètodes de l'objecte que han d'executar-se en exclusió mútua.
- Abans d'executar un mètode sincronitzat cal **competir** per a aconseguir **el bloqueig de l'objecte** al qual pertany.

Mètodes sincronitzats II

Tots els blocs definits synchronized en una classe, formen part del mateix bloqueig **quan es crea un objecte**.

- Per exemple, si hi ha dos mètodes (*metode1* i *metode2*) sincronitzats, només un fil pot accedir als mètodes del mateix objecte.
- Si el *fil1* accedeix a *metode1*, un altre *fil2* no pot accedir ni a *metode1* (obvi) ni a *metode2* fins que *fil1* isca de *metode1* *(sempre pensant que fil1 i fil2 accedeixen al mateix objecte)*.
- Si *fil1* accedeix a un objecte i *fil2* a un altre objecte, encara que siga de la mateixa classe, no hi ha bloquejos.

Pràctica Compte

Realitza modificacions en les classes perquè el mètode adequat s'execute de manera atòmica i indivisible.

Recorda aplicar synchronized a un mètode determinat:

```
public synchronized void metode() {
    }
```

Execució

L'execució després de les modificacions ha de ser:

```
[ramon@nowhere PSP_Setmana 4]$ java CompartirInf3
Anna va a treure diners (actual es: 40)
  Anna treu => : 10. Saldo Actual: 30
Joan va a treure diners (actual es: 30)
  Joan treu => : 10. Saldo Actual: 20
Anna va a treure diners (actual es: 20)
  Anna treu => : 10. Saldo Actual: 10
Anna va a treure diners (actual es: 10)
  Anna treu => : 10. Saldo Actual: 0
Joan no pot treure diners. No hi ha saldo. Saldo Actual: 0
Anna no pot treure diners. No hi ha saldo. Saldo Actual: 0
Joan no pot treure diners. No hi ha saldo. Saldo Actual: 0
Joan no pot treure diners. No hi ha saldo. Saldo Actual: 0
```